THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD

Department of Software Engineering

LAB 03 REPORT — SOFTWARE CONSTRUCTION & DEVELOPMENT (LAB)

COURSE TITLE	**DIGITAL IMAGE PROCESSING**
SEMESTER	**Fall 2026**
SUBMITTED TO	**Engr. Fahad Nisar**
SUBMITTED BY	**Fatima Arshad**
ROLL NUMBER	**2024-SE-24**
DEPARTMENT	**Software Engineering**

DEPARTMENT OF SOFTWARE ENGINEERING

Table of Contents

- 1. Introduction
- 2. Objectives
- 3. Tools and Libraries
- 4. Input Image and Validation
- 5. Reusable Transform Function
- 6. Geometric Transformations
- 7. Interpolation Comparison
- 8. Input Validation
- 9. Identity Tests
- 10. Results and Discussion
- 11. Important Concepts
- 12. Conclusion

1. Introduction

Geometric transformations change the size, position, orientation, or visible region of an image. They are common operations in digital image processing and computer vision. In this lab, a reusable Python function named transform() is developed to perform scaling, cropping, flipping, translation, and rotation using custom parameters. The original image is preserved and the transformed results are saved separately.

2. Objectives

- Load an image and validate that it exists, decodes correctly, and has non-zero dimensions.
- Display image width, height, and NumPy array shape.
- Create one reusable transform() function for all required operations.
- Accept custom parameters and validate them inside the function.
- Demonstrate all geometric operations, including boundary and identity cases.
- Compare four interpolation methods and save transformed results.

3. Tools and Libraries

Tool / Library	Purpose
Python	Programming language for the implementation.
Jupyter Notebook	Interactive environment for code and output.
NumPy	Image arrays and numerical operations.
OpenCV	Image loading and geometric transformations.
Matplotlib	Displaying original and transformed images.
Pillow	Saving RGB image results.

4. Input Image and Validation

The project contains a generated RGB sample image named input_image.png so the notebook can run immediately. The image-loading function checks that the file exists, can be decoded, and has valid dimensions. OpenCV loads images in BGR format; therefore, the image is converted to RGB for consistent display.

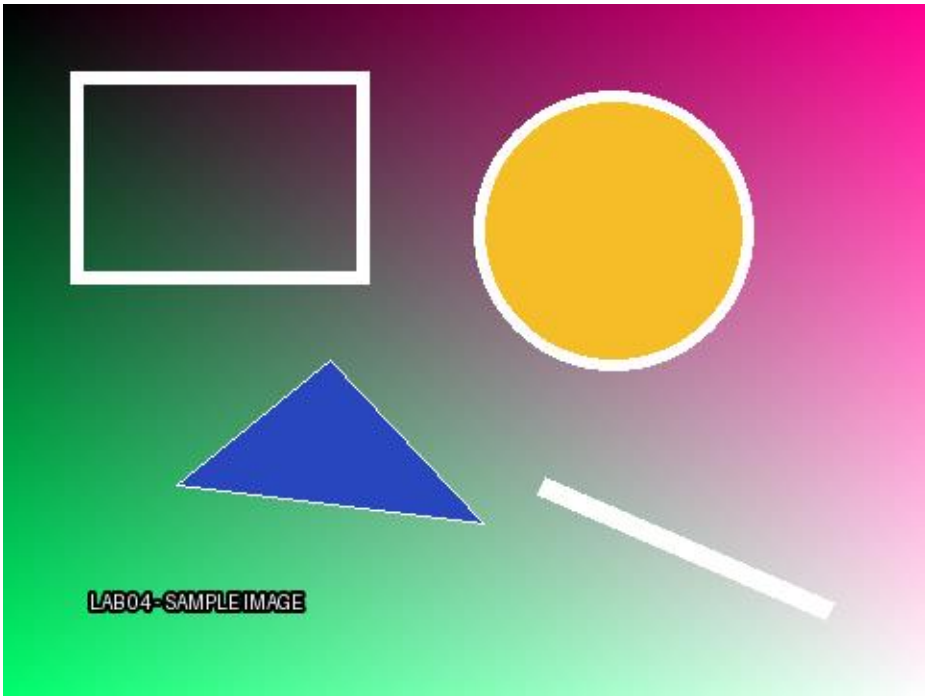


Figure 1. Sample RGB input image used in the lab.

The sample image dimensions are 480 pixels in width and 360 pixels in height. Its expected NumPy shape is (360, 480, 3), where the last dimension represents the three colour channels.

5. Reusable Transform Function

The transform() function is the main part of the program. It accepts an image, an operation name, and custom keyword parameters. It validates the image and parameters, performs the selected operation, and returns the transformed image. The original image is not modified.

Operation	Custom inputs	Validation rules
Scale	sx, sy, interpolation	Finite positive scale factors; supported interpolation; output dimensions at least one pixel.
Crop	left, top, right, bottom	Integer coordinates within image boundaries; right and bottom are exclusive.
Flip	direction	Only horizontal, vertical, or both are accepted.
Translate	tx, ty	Finite positive, negative, and zero offsets are accepted.
Rotate	angle, expand	Finite angle; expand must be True or False.

A maximum output size of 25 million pixels is enforced to avoid excessive memory use. Numeric values such as NaN and infinity are rejected. Unknown operations and missing required parameters also produce clear error messages.

6. Geometric Transformations

6.1 Scaling / Resizing

Scaling changes the image width and height. The sx parameter controls horizontal scaling and sy controls vertical scaling. Equal factors preserve the aspect ratio, while different factors stretch or compress the image.

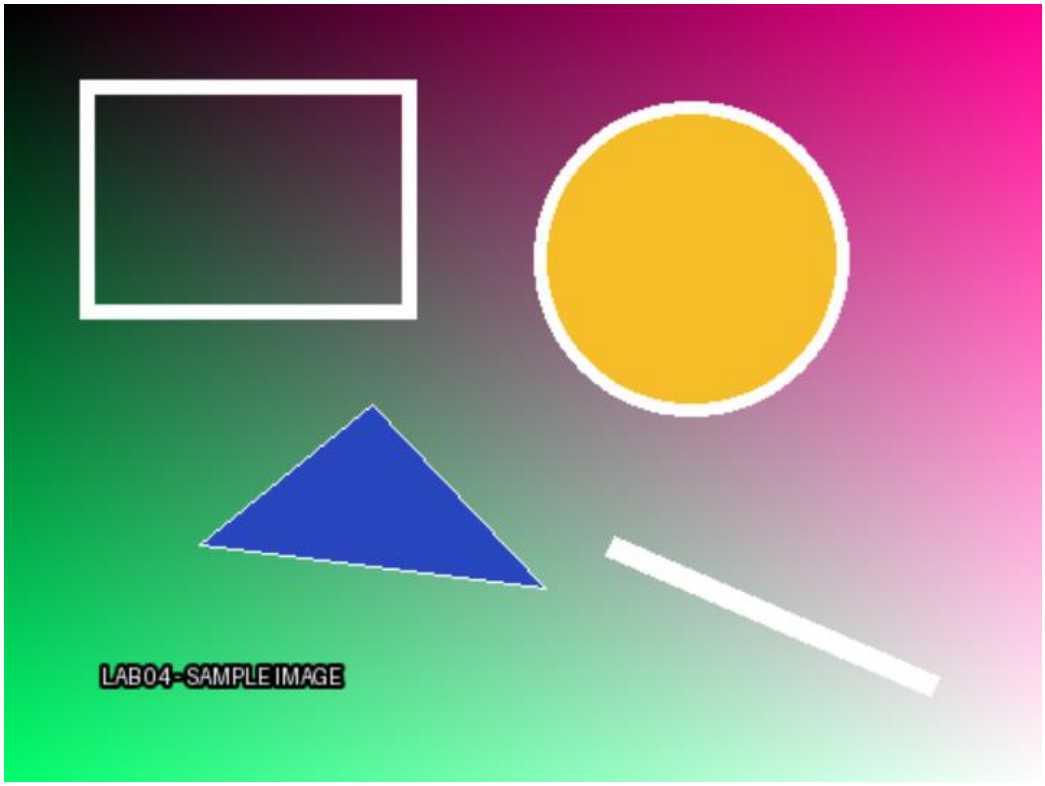


Figure 2. Proportional enlargement: sx = 1.5, sy = 1.5.

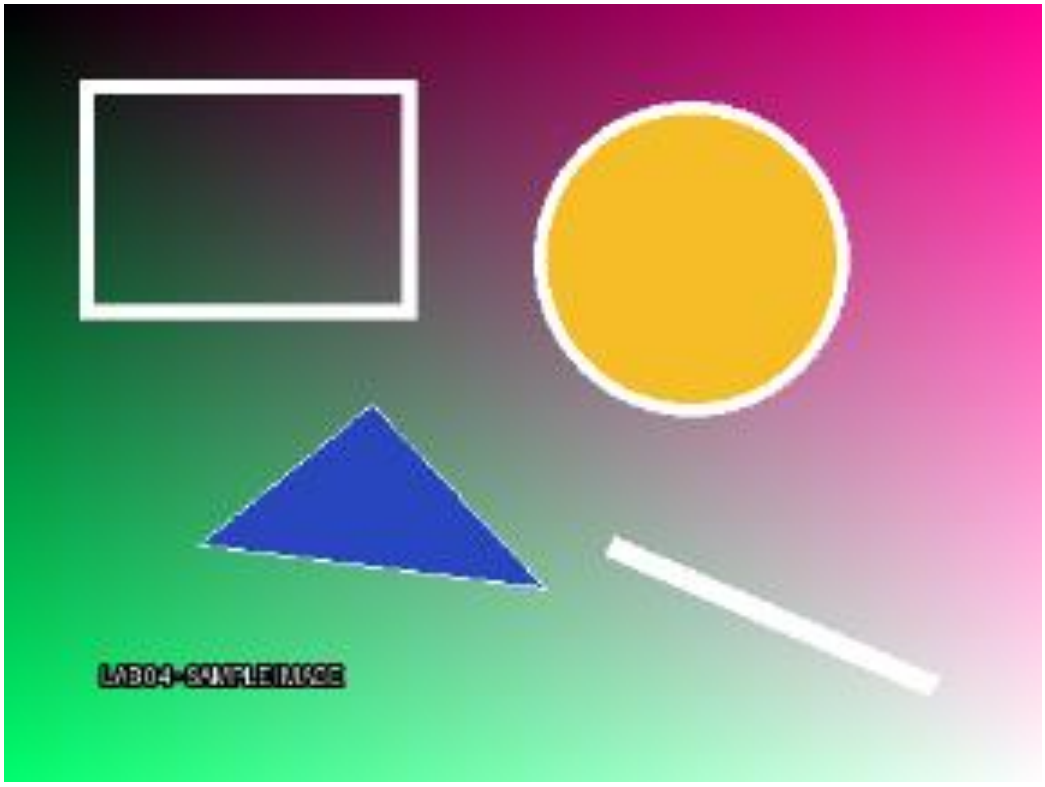


Figure 3. Proportional reduction: sx = 0.6, sy = 0.6.

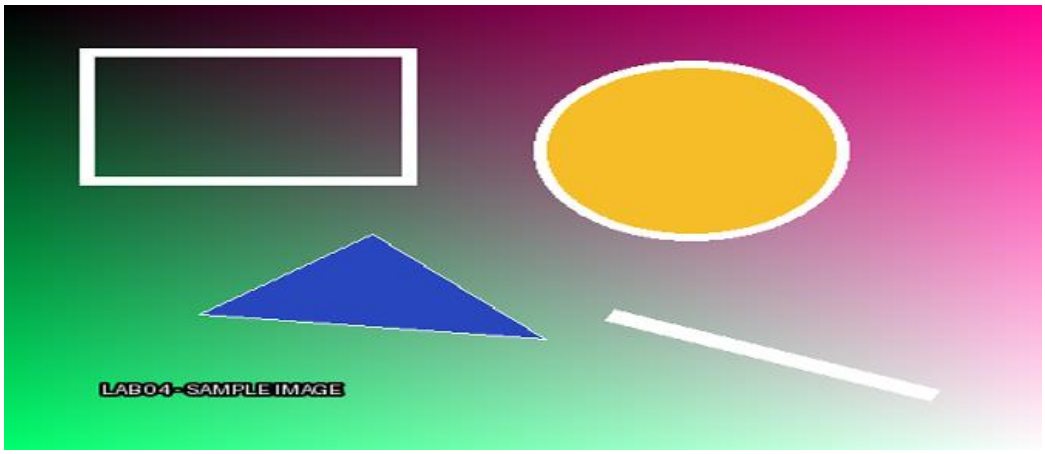


Figure 4. Non-proportional scaling: sx = 1.4, sy = 0.8.

6.2 Cropping

Cropping extracts a rectangular region from the image. Valid coordinates must satisfy $0 \leq \text{left} < \text{right} \leq \text{width}$ and $0 \leq \text{top} < \text{bottom} \leq \text{height}$. The right and bottom boundaries are exclusive.

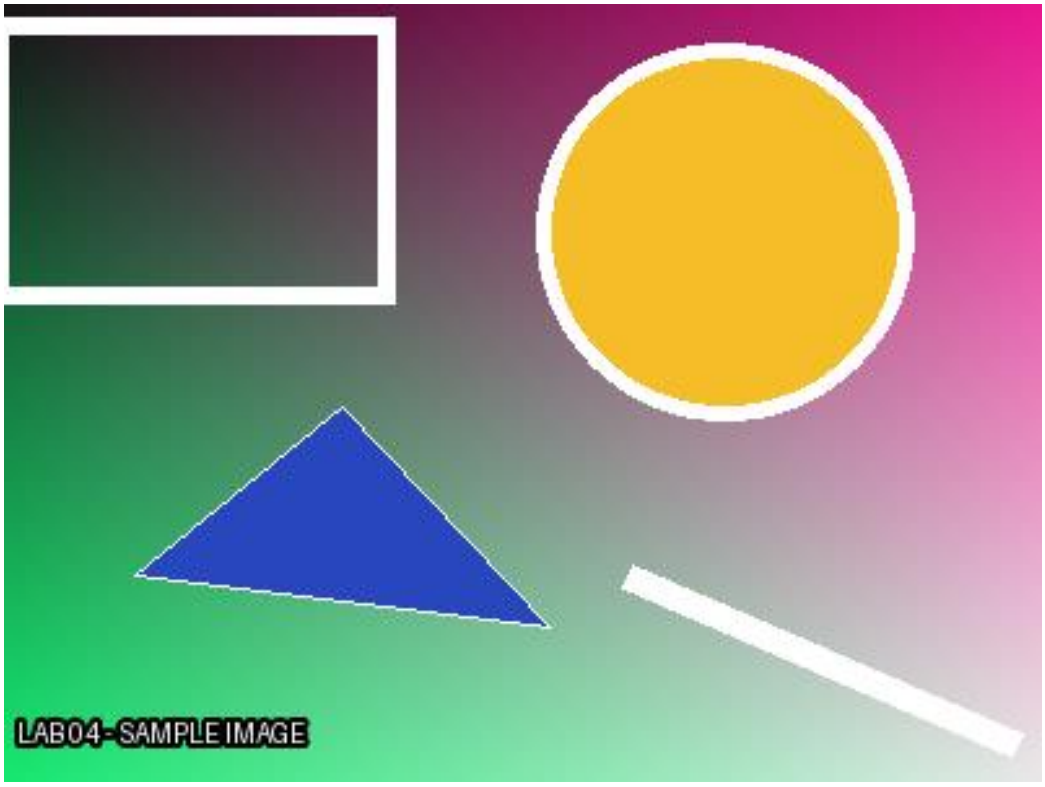


Figure 5. Cropped image using custom coordinates.

6.3 Flipping

A horizontal flip reverses image columns. A vertical flip reverses image rows. A flip in both directions reverses both rows and columns.

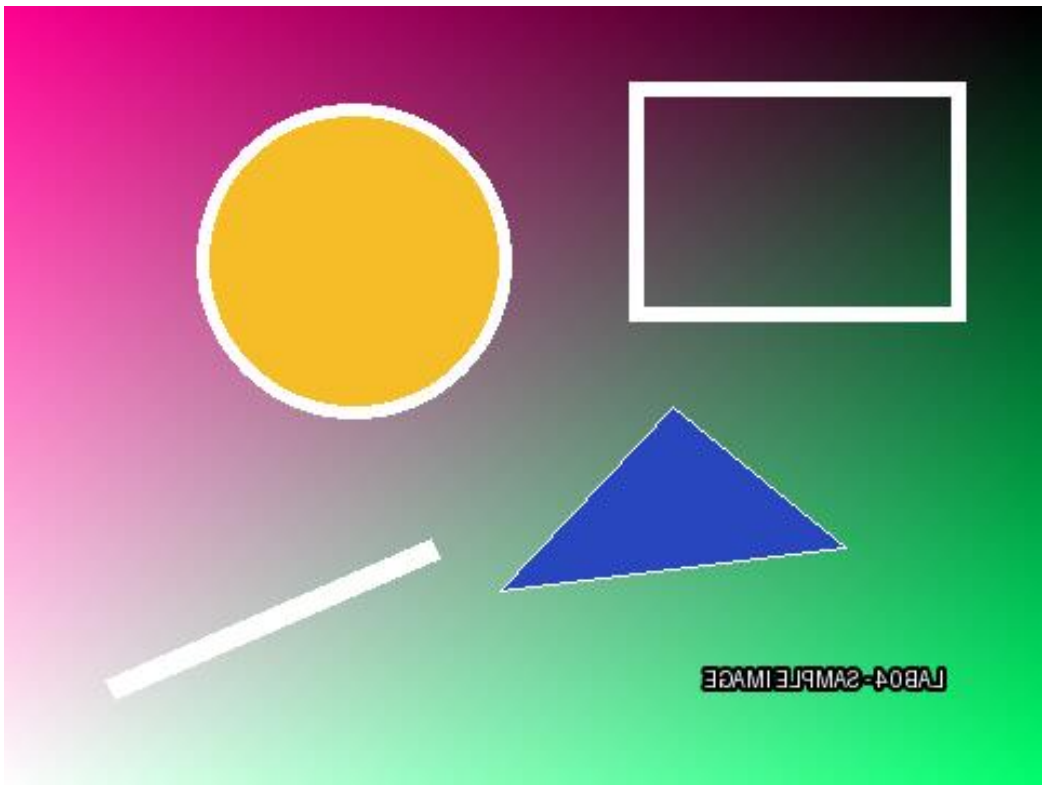


Figure 6. Horizontal flip.

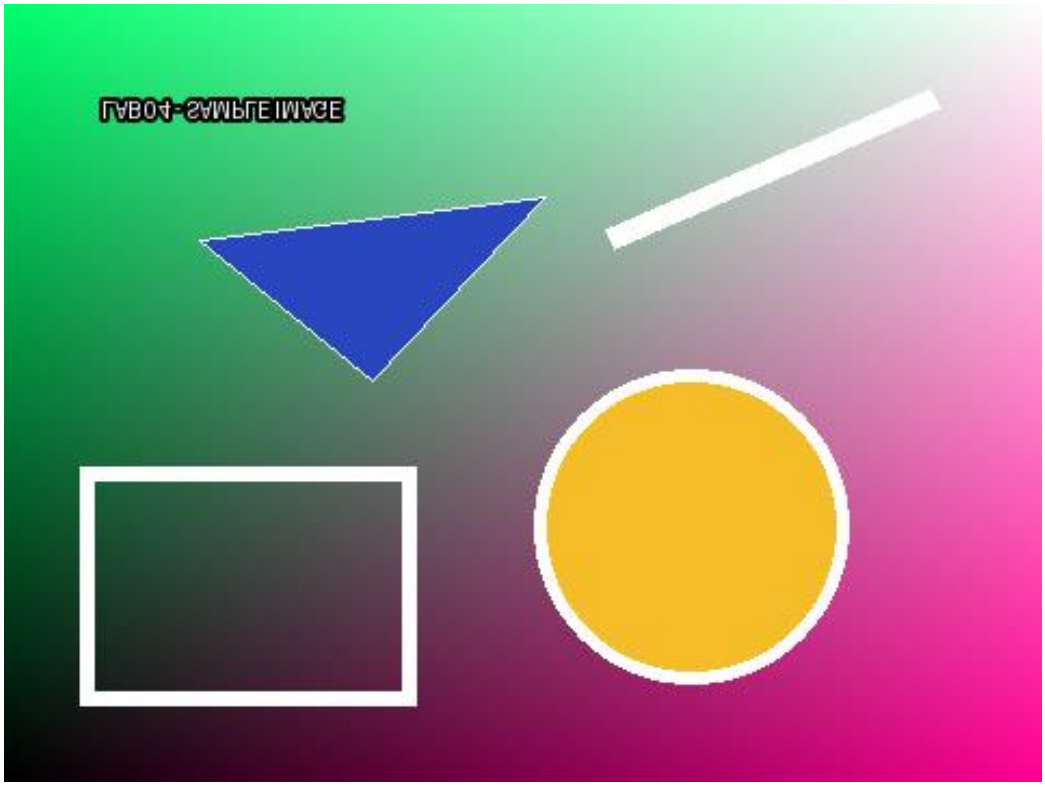


Figure 7. Vertical flip.

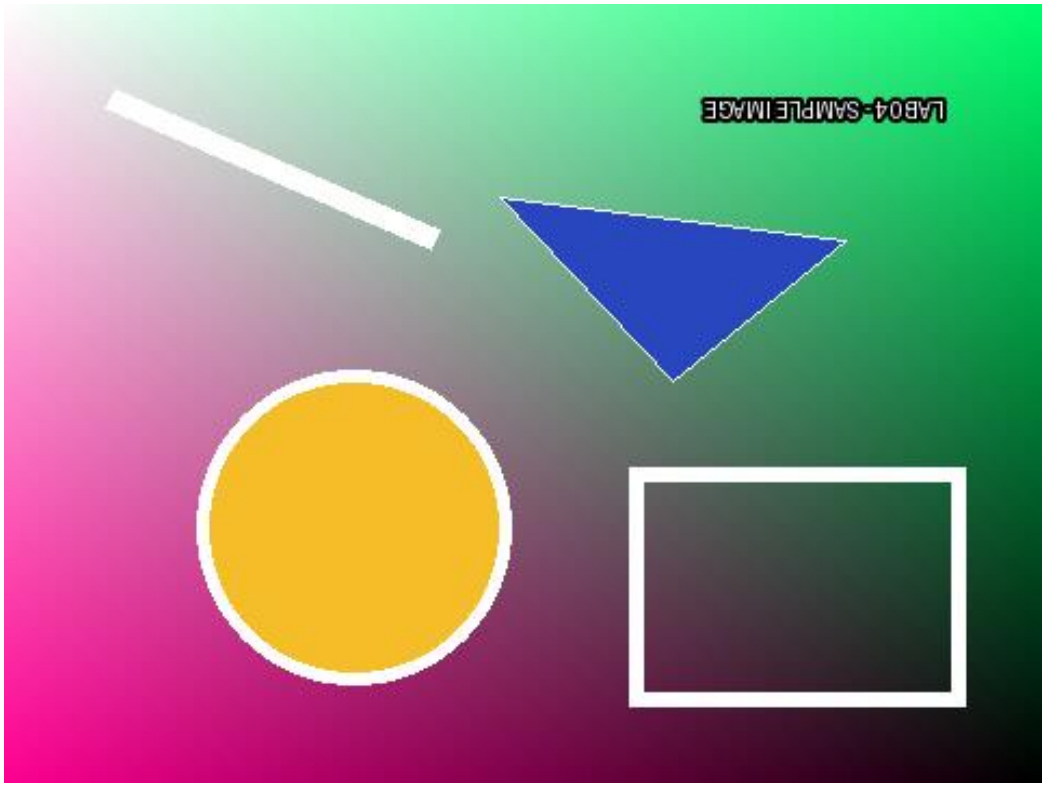


Figure 8. Flip in both directions.

6.4 Translation

Translation moves pixels using horizontal offset tx and vertical offset ty. Positive tx moves right and negative tx moves left. Positive ty moves down and negative ty moves up. The output canvas remains the original size. Pixels shifted outside the canvas are clipped, and newly exposed areas are filled with black.

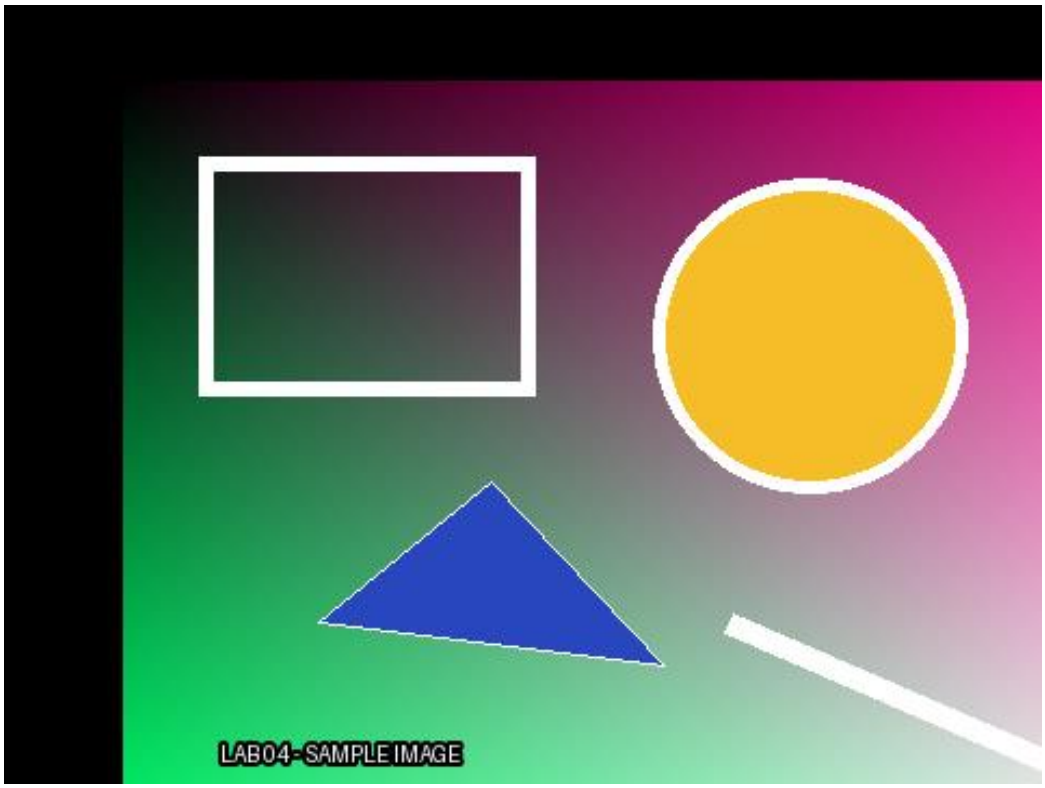


Figure 9. Translation: tx = 55, ty = 35.

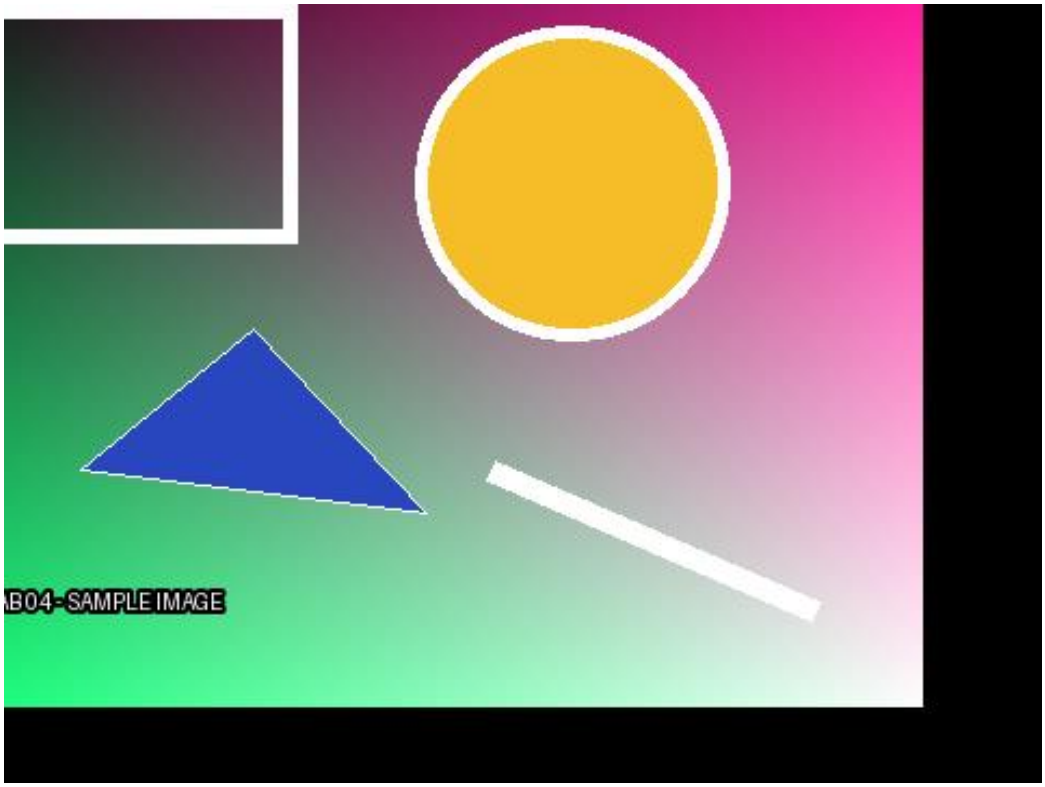


Figure 10. Translation: tx = -55, ty = -35.

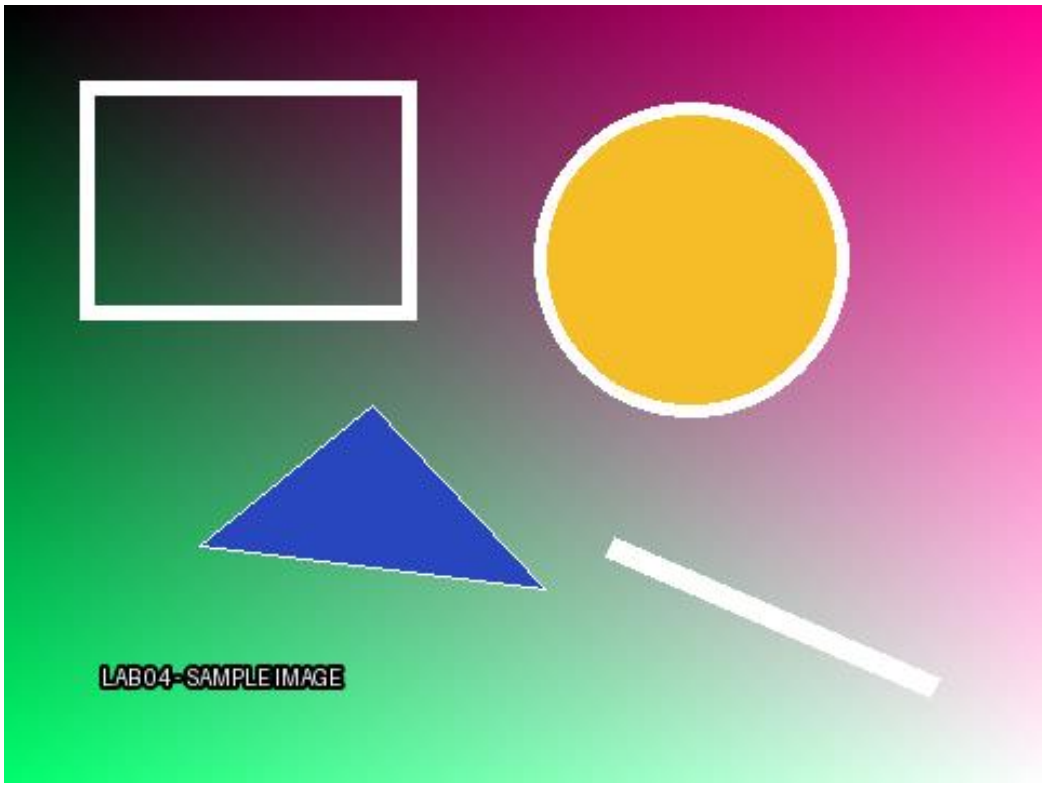


Figure 11. Zero translation: tx = 0, ty = 0.

6.5 Rotation

Rotation turns the image around its centre. Positive angles rotate counter-clockwise in the displayed image coordinate system used by OpenCV, while negative angles rotate clockwise. Without canvas expansion, the original dimensions are retained and corners may be clipped. With expansion, the output canvas is enlarged to fit the rotated image.

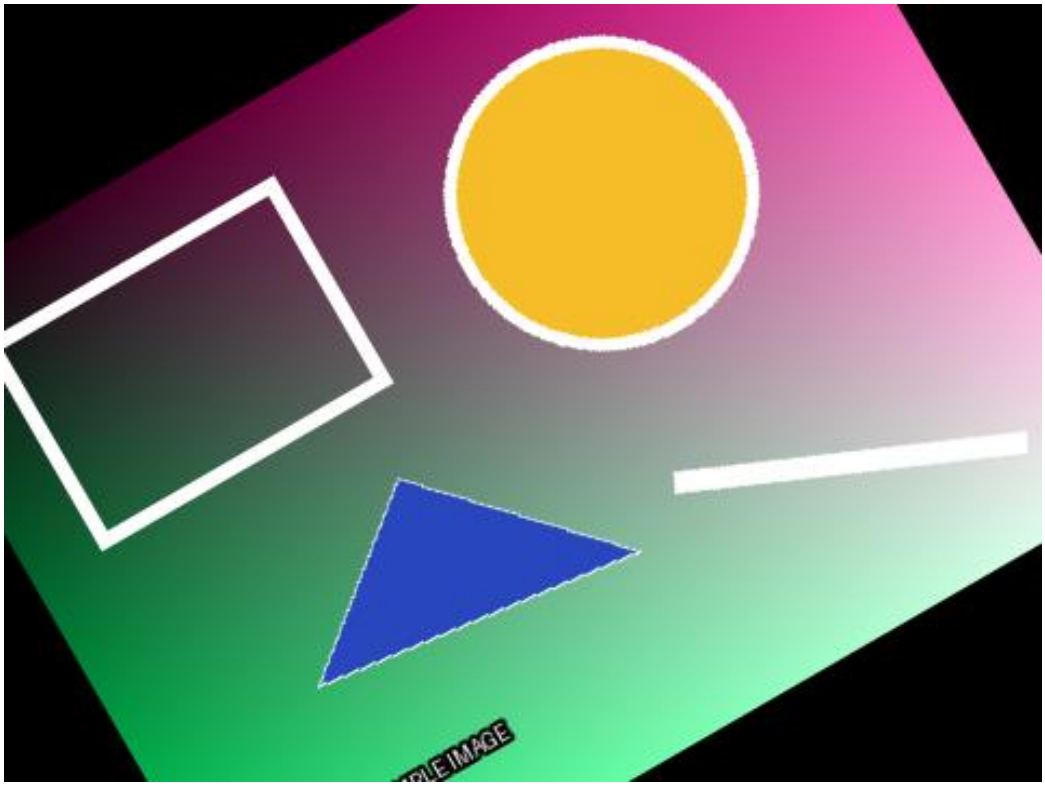


Figure 12. Positive rotation: +30 degrees, fixed canvas.

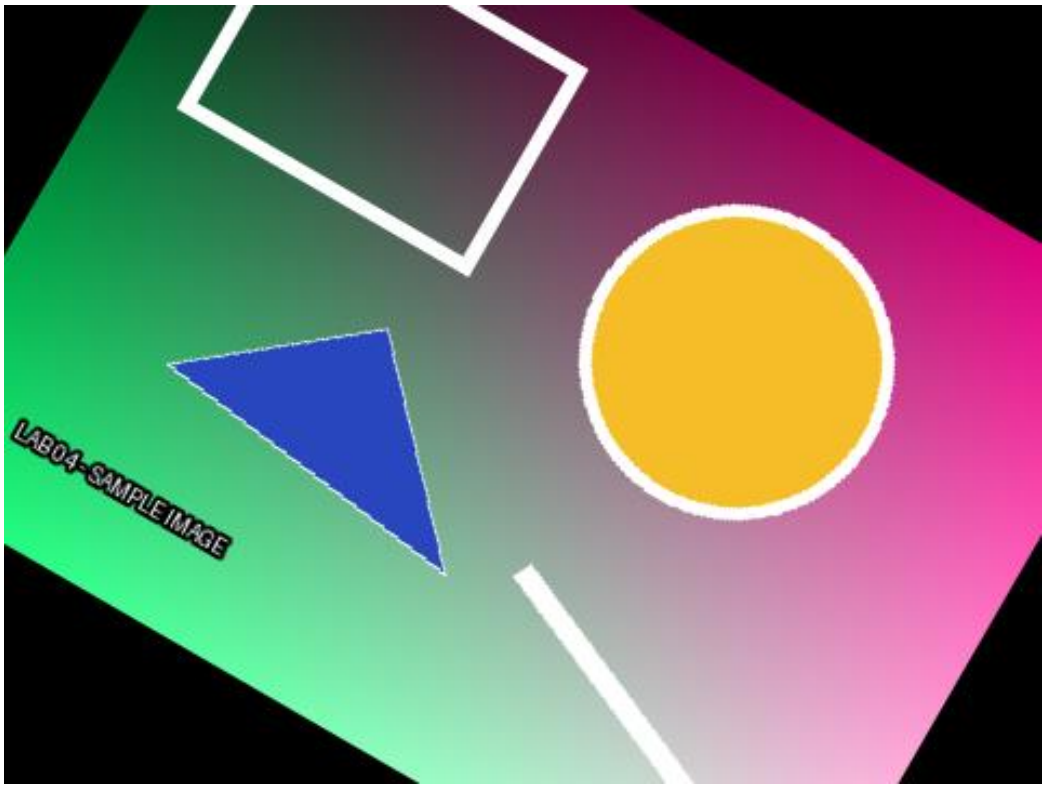


Figure 13. Negative rotation: -30 degrees, fixed canvas.

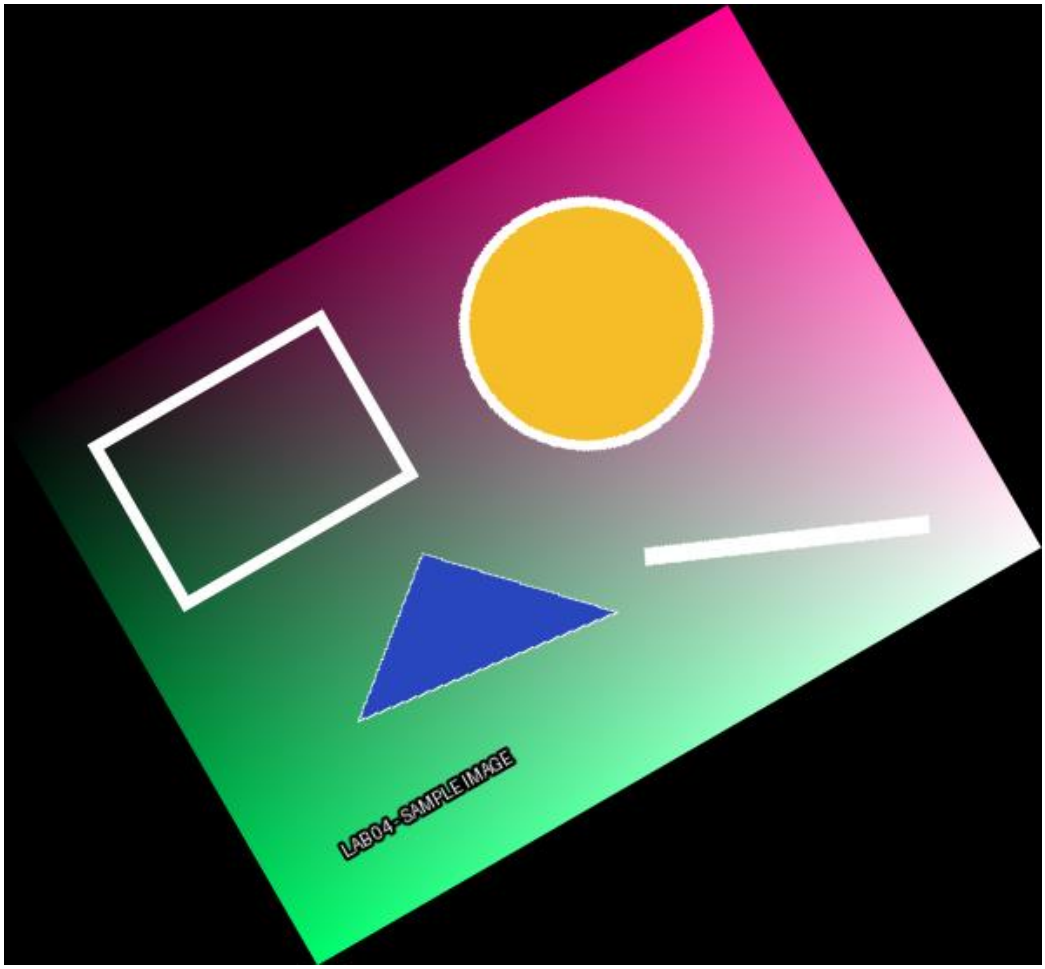


Figure 14. Positive rotation: +30 degrees, expanded canvas.

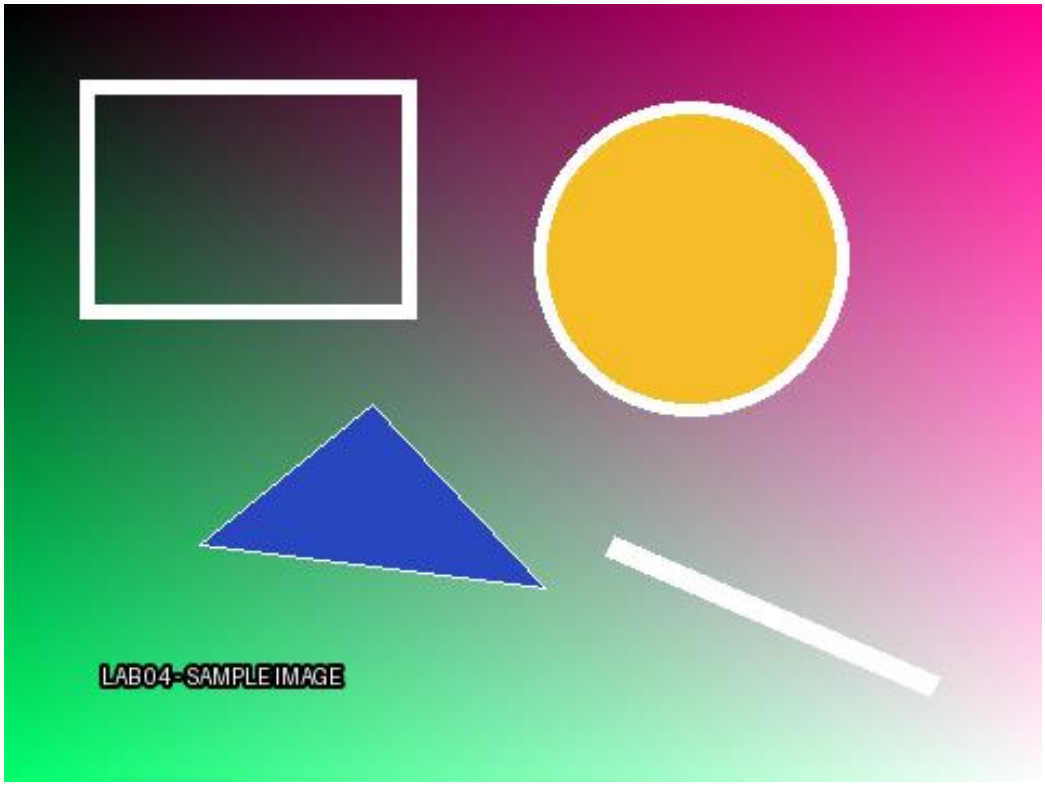


Figure 15. Zero-degree rotation.

7. Interpolation Comparison

Interpolation estimates pixel values when an image is resized. The same cropped patch was enlarged by a factor of 2 using four interpolation methods, allowing a fair visual comparison.

Method	Observation
Nearest-neighbour	Copies the nearest pixel. It is fast but can look block-like.
Bilinear	Averages nearby pixels and usually looks smoother, but may appear soft.
Bicubic	Uses a larger neighbourhood than bilinear and often gives smoother detail.
Lanczos	Can preserve fine detail well, although slight ringing may appear near sharp edges.



Figure 16. Nearest-neighbour interpolation.



Figure 17. Bilinear interpolation.



Figure 18. Bicubic interpolation.



Figure 19. Lanczos interpolation.

8. Input Validation and Error Handling

Validation is performed inside transform(), so invalid parameters are rejected even when the function is called directly. The notebook demonstrates rejected inputs for each operation.

Operation	Rejected input example	Expected handling
Scale	Zero scale factor	Reject; factors must be greater than zero.
Crop	Left coordinate greater than right	Reject; crop boundaries are invalid.
Flip	Unsupported direction: diagonal	Reject; only three supported choices.
Translate	NaN offset	Reject; offset must be finite.
Rotate	Nonnumeric angle: thirty	Reject; angle must be numeric and finite.
Additional	Unknown operation, missing parameter, unsupported interpolation, infinity	Reject with a clear error message.

The notebook also provides a custom-input prompt function. It asks the user for the operation and required values. If invalid text or values are entered, an error message is displayed and the function can be run again.

9. Identity Tests

An identity transformation should leave the image unchanged. The notebook checks scaling with factors of one, translation with zero offsets, and rotation by zero degrees. Each result is compared with the original using both array shape and pixel values.

Identity case	Expected behaviour	How actual result is checked
sx = 1, sy = 1	Image remains unchanged	Shape and pixel values compared.
tx = 0, ty = 0	Image remains unchanged	Shape and pixel values compared.
angle = 0°	Image remains unchanged	Shape and pixel values compared.

The notebook prints Pass or Fail for each identity case when executed. Keep those actual outputs visible in the submitted notebook.

10. Results and Discussion

- All five required transformations are implemented through one reusable function.
- Scaling demonstrates enlargement, reduction, and non-proportional resizing.
- Cropping validates integer coordinates and exclusive right/bottom boundaries.
- All three flip directions are demonstrated.
- Translation supports positive, negative, and zero offsets while keeping the original canvas size.
- Rotation supports positive, negative, and zero angles with optional canvas expansion.
- Invalid inputs are rejected with readable messages, and transformed images are saved separately.

11. Important Concepts

11.1 Image Coordinates and Array Indexing

Image coordinates are commonly written as (x, y), where x is the horizontal column and y is the vertical row. NumPy uses [row, column], so a pixel is accessed as image[y, x].

11.2 Aspect-Ratio Distortion

Aspect ratio is width divided by height. Equal scale factors preserve it. Different horizontal and vertical factors change the ratio and may stretch or compress objects.

11.3 Rotation Canvas Expansion

A fixed-size canvas may cut off parts of a rotated image. Canvas expansion creates a larger output area so more of the rotated image can fit.

11.4 Translation Clipping

Pixels moved beyond the original image boundary are clipped. Newly exposed areas are filled with black in this implementation.

12. Conclusion

This lab implemented geometric image transformations using Python, OpenCV, NumPy, Matplotlib, and Pillow. A reusable transform() function was created to perform scaling, cropping, flipping, translation, and rotation with custom parameters. Input validation rejects invalid operations and parameter values. The notebook demonstrates the required operations, compares four interpolation methods, checks identity cases, and saves transformed images separately. The original image remains unchanged.

The practical work demonstrates how transformations affect image size, orientation, position, and visible content. It also shows why validation is necessary for reliable image-processing programs.